



Lecture 12: Programmatic Component Instantiation

INTERVIEW QUESTION

How do you instantiate a compiled Svelte component programmatically within vanilla JavaScript?

When you build a modern Svelte application, you spend 99% of your time entirely inside the Svelte ecosystem (`.svelte` files, runes, components). But the browser itself has no idea what "Svelte" is. It only understands plain HTML, CSS, and vanilla JavaScript.

Because of this, every single Svelte application needs an **ignition switch**—a boundary layer where standard JavaScript reaches into the Svelte universe, grabs the root component, and physically attaches it to a raw HTML element (like `<div id="app"></div>`).

In Svelte 5, you accomplish this using the imperative component API, specifically the `mount` function. Without it, your Svelte app is just inert code sitting in memory.

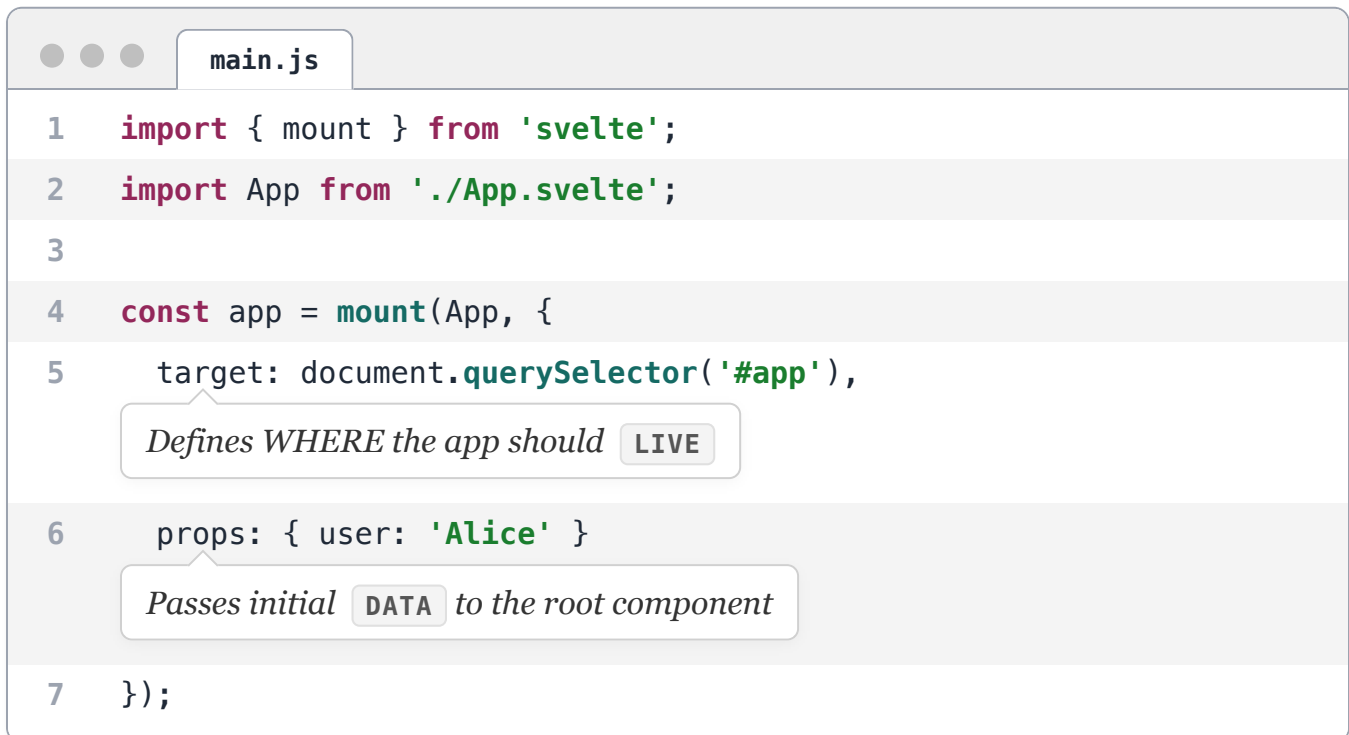
The functional approach to mounting

KEY TAKEAWAY

Svelte 5 components are lightweight functions, not heavy object-oriented classes.

In the past, Svelte components were compiled into JavaScript classes, and you instantiated them using the `new` keyword (e.g. `new App({ target })`). Because Svelte 5 shifted to a much faster, signal-based reactivity engine under the hood, components are no longer class instances. They are functions that set up a reactive graph.

Because they are no longer classes, you can no longer use `new`. Instead, Svelte provides a `mount` function to bring your component to life.



```
1  import { mount } from 'svelte';
2  import App from './App.svelte';
3
4  const app = mount(App, {
5    target: document.querySelector('#app'),
6    props: { user: 'Alice' }
7  });
```

Defines WHERE the app should LIVE

Passes initial DATA to the root component

The `mount` function takes two arguments:

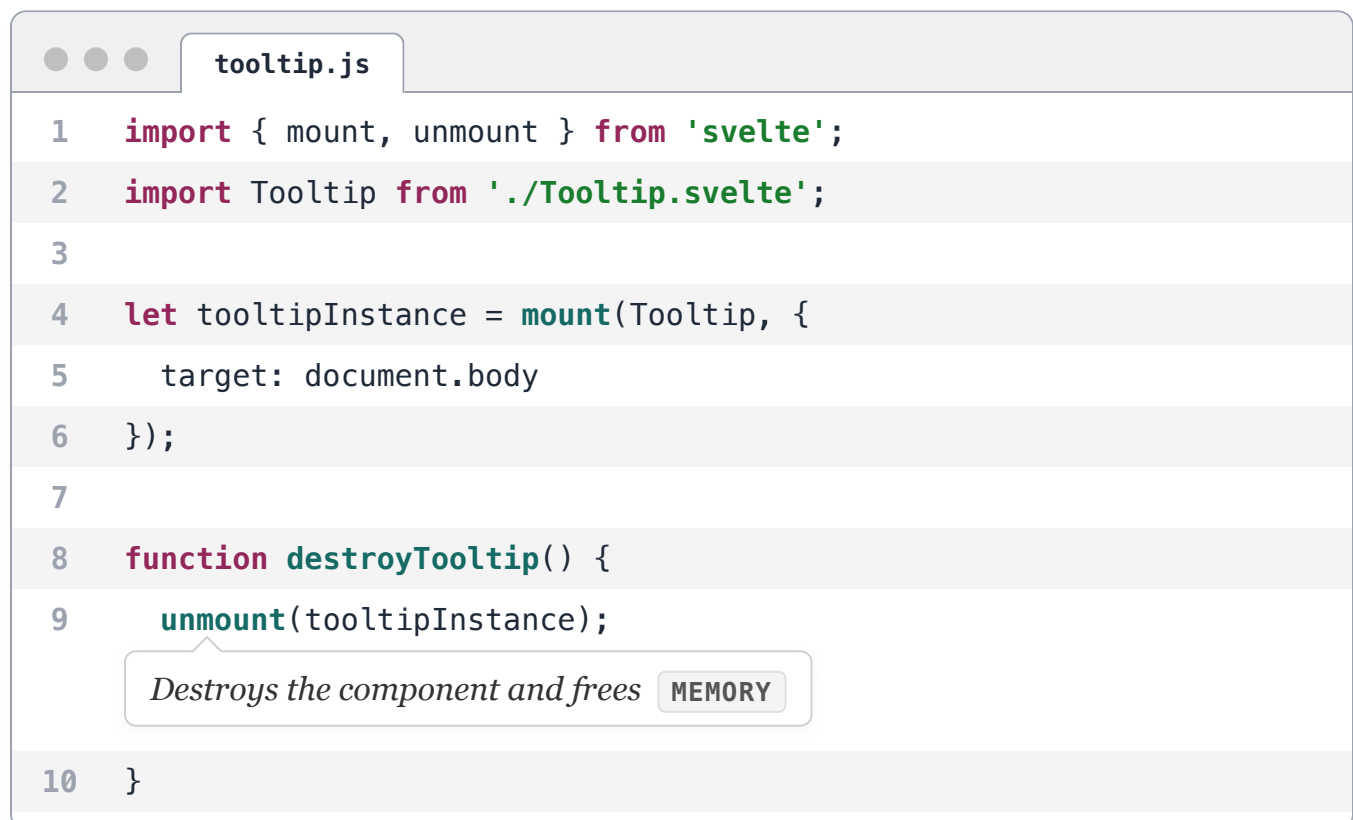
1. The compiled component itself (in this case, `App`).
2. An options object that **must** contain a `target` (the physical DOM node where the component will be injected), and optionally `props` (the data you want to hand to the component when it wakes up).

Tearing down the app

INTERVIEW TIP

To impress the interviewer: If asked about programmatic components, do not explain only how to start them; explain how to *destroy* them. Mentioning `unmount` proves you understand the full lifecycle and how to prevent memory leaks in long-running single-page applications.

As `mount` brings a component to life, Svelte 5 provides an `unmount` function to destroy it. This is critical when you are mounting components dynamically inside a larger non-Svelte application, or building something like a temporary tooltip that needs to vanish cleanly.



```
1  import { mount, unmount } from 'svelte';
2  import Tooltip from './Tooltip.svelte';
3
4  let tooltipInstance = mount(Tooltip, {
5    target: document.body
6  });
7
8  function destroyTooltip() {
9    unmount(tooltipInstance);
10 }
```

Destroys the component and frees **MEMORY**

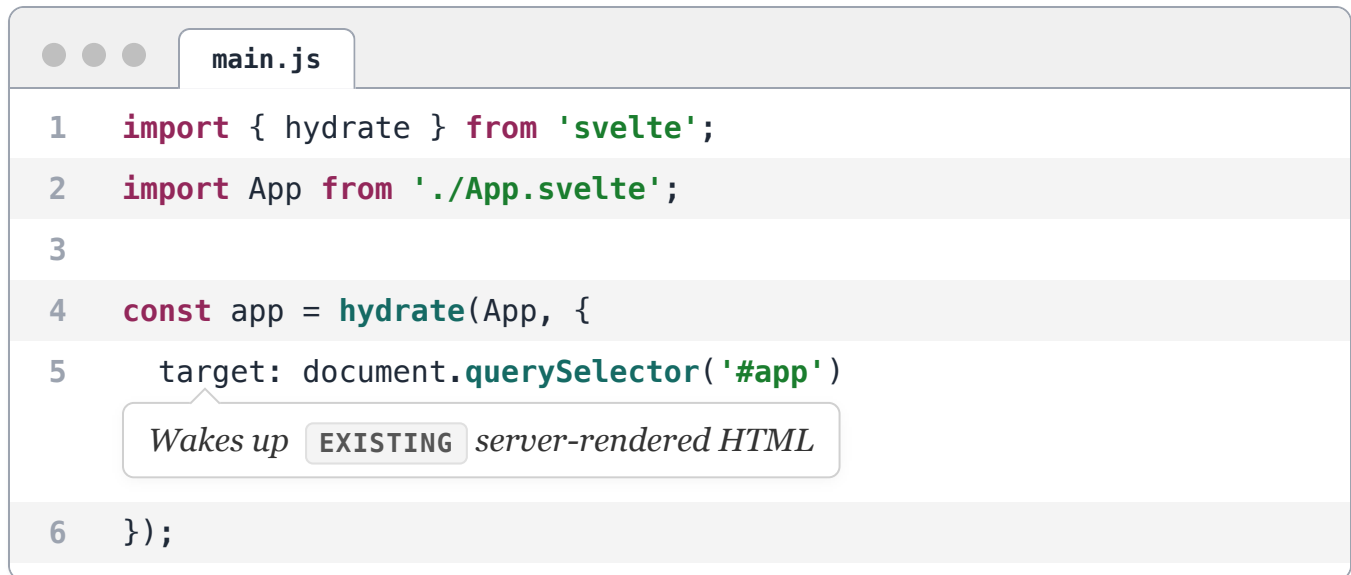
When you call `unmount`, Svelte safely removes the elements

from the DOM and tears down the component's internal reactive graph, preventing memory leaks.

Hydrating existing HTML

Sometimes, your server has already rendered the initial HTML of your Svelte component and sent it to the browser. In this scenario, you do not want `mount` to create brand new DOM elements from scratch; you want Svelte to attach its reactive listeners to the HTML that is already there.

For this, Svelte provides `hydrate` :



```
main.js
1  import { hydrate } from 'svelte';
2  import App from './App.svelte';
3
4  const app = hydrate(App, {
5    target: document.querySelector('#app')
6  });
```

Wakes up EXISTING server-rendered HTML

`hydrate` walks through the existing DOM inside the `target` and carefully "wakes it up" by attaching event listeners and reactivity, making the page interactive without the expensive cost of redrawing it.

Summary

To programmatically instantiate a Svelte 5 component in vanilla JavaScript, import the `mount` function from `svelte` and pass it your component along with a target DOM element. Avoid the legacy `new Component()` syntax, as Svelte 5 components are now optimized functions, not classes. Use `unmount` to clean up, and `hydrate` if the HTML was already rendered by the server.

Beyond the basics

Frequency Rating System ★★★ = *Very common
gotcha / pattern* ★★☆☆ = *Common gotcha / pattern*
★☆☆ = *Niche / Advanced pattern*

★★★ `mount` and `hydrate` do NOT run effects during the call

The lecture shows `mount` and `hydrate` as the entry points, but it leaves out the most surprising behavior in the docs: unlike calling `new App(...)` in Svelte 4, **effects do not run during `mount`** — and the same is true of `hydrate`. Effects, including `onMount` callbacks and action functions, are queued and run later. This matters the moment you write a test or a script that asserts on the DOM right after `mount` returns: nothing has happened yet, because the effects that would have built up the reactive graph are still pending. If you need them to run synchronously — in a test, for instance — call `flushSync()` immediately after `mount` to force the pending effects to flush. See [documentation 2026 June/svelte-docs/06-runtime/04-imperative-component-api.md](#) (`mount` and `hydrate` sections).

★★★ `unmount` can play outro transitions before removal

The lecture presents `unmount` as a one-shot teardown. The docs reveal an option it did not mention: pass `{ outro: true }` and Svelte will play any outbound transitions on the component before removing it from the DOM. In that mode `unmount` returns a `Promise` that resolves once the transitions complete, so you can `await` the teardown — useful when a modal or toast needs to animate out before the next step runs. Without `outro`, `unmount` returns immediately. See [documentation 2026 June/svelte-docs/06-runtime/04-imperative-component-api.md](#) (`unmount` section).

★★☆ `render` is the server-side counterpart to `mount`

`mount` and `hydrate` are the client-side pair, but the docs describe a third sibling: `render`, imported from `svelte/server`. It is only available on the server (and only when compiling with the `server` option). Instead of attaching to a DOM node, it takes a component and returns an object with two strings — `body` (the HTML for the document body) and `head` (the HTML for the document head). This is the function that produces the server-rendered HTML which `hydrate` later wakes up, so understanding `render` completes the SSR loop: `render` on the server produces HTML, the browser receives it, and `hydrate` attaches interactivity to it without rebuilding the DOM. See [documentation 2026 June/svelte-docs/06-runtime/04-imperative-component-api.md](#) (`render` section).

★★☆ You can mount many components, and mount from inside your app

The lecture's examples mount a single root `App`, but the docs make a point worth carrying: you can mount multiple components on the same page, and you can call `mount` from *inside* a running Svelte application. The canonical use case is a tooltip component that mounts itself onto the currently hovered element — a pattern that produces DOM nodes living outside the normal component tree (often attached directly to `document.body`). The lecture's `Tooltip` example gestures at this, but the docs state it as an explicit capability rather than an incidental one. See [documentation 2026 June/svelte-docs/06-runtime/04-imperative-component-api.md](#) (`mount` section).

☆☆☆ **hydratable**
data: the low-level API
for SSR-safe async
values

Separate from the `hydrate` function is a rune-shaped API named `hydratable` (imported from `svelte`) that solves a specific SSR pitfall: when a component awaits data on the server, the client has to redo that same async work during hydration, blocking interactivity until it finishes.

`hydratable(key, fn)` runs `fn` once on the server, serializes the result into the page's `<head>`, and returns the cached value during hydration instead of re-running the function. You will rarely reach for this directly — it powers data-fetching libraries and SvelteKit's remote functions behind the scenes — but knowing it exists explains how production Svelte apps avoid double-fetching during hydration. See [documentation 2026 June/svelte-docs/06-runtime/05-hydratable.md](#).